

STUDENT PLACEMENT MANAGEMENT SYSTEM



NAME : ELLUR GOWRAV

REGISTRATION NO :25BAI10008

COURSE NAME: PROGRAMMING IN JAVA



INTRODUCTION

The **Student Placement Management System** is a Java-based console application designed to simplify and organize the placement process for students, companies, and placement coordinators. The system provides a structured platform for maintaining student profiles, managing companies and placement drives, checking student eligibility, processing applications, tracking selection status, and generating placement-related reports.

The system is developed using core Java programming concepts, particularly object-oriented programming, collections, exception handling, enumerations, Java streams, date and time APIs, and modular separation of responsibilities. The application represents real-world entities such as students, companies, placement drives, and applications as Java classes and manages their relationships through repositories and service classes.

The main purpose of the system is to automate important placement activities and reduce manual processing. A placement drive can define eligibility criteria such as minimum CGPA, maximum number of backlogs, eligible branches, graduation years, and required skills. Students can apply for suitable drives, after which the system evaluates their eligibility and manages the application through different stages such as eligible, shortlisted, selected, or rejected.

The project demonstrates how Java can be used to develop a structured real-world management application. It also provides reporting features such as placement percentage, placement count by branch, average package by branch, company-wise selections, and identification of top package students.

PROBLEM STATEMENT

Managing student placement activities manually can become difficult when the number of students, companies, placement drives, and applications increases. Placement coordinators need to **maintain student information, company details, job requirements, eligibility criteria, application status**, and final selection information. Manual management of these activities can result in duplicate applications, incorrect eligibility decisions, difficulty in tracking application status, and challenges in generating placement statistics.

The proposed Student Placement Management System addresses these problems by providing a centralized Java-based application for managing the major activities involved in a campus placement process. **The system maintains student profiles containing academic information, branch, graduation year, CGPA, backlogs, and skills.** Companies and their placement drives can also be recorded along with job information and eligibility requirements.

The system automatically checks whether a student satisfies the eligibility criteria of a placement drive. It also prevents duplicate applications and invalid application operations. Applications can progress through defined stages from application submission to eligibility, shortlisting, selection, or rejection. Finally, the system provides **placement reports and statistics** to help users understand placement outcomes.

Functional Requirements

The first module is Student Management. It allows the system to create and maintain student profiles containing student ID, name, branch, graduation year, CGPA, number of backlogs, and technical skills. Students can also be displayed and searched using their **unique student ID**. The second module is Company Management. This module allows companies to be added to the system with information such as company ID, company name, industry sector, and HR contact email. The system also provides functionality for displaying registered companies.

The third module is **Placement Drive Management**. A placement drive contains information such as job title, offered package, drive date, minimum CGPA, maximum allowed backlogs, eligible branches, eligible graduation years, and required skills. Placement drives can be created and listed through the application.

The fourth module is **Eligibility Management**. The system evaluates a student's eligibility against the requirements of a placement drive. The eligibility check considers CGPA, branch, graduation year, backlog count, required skills, and whether the drive is still open.

The fifth module is **Application Management**. Students can apply for placement drives. The system prevents duplicate applications and prevents applications to closed drives. Application records contain the student, placement drive, application status, application date, and remarks. The sixth module is **Selection Management**. Applications that pass eligibility checks can be shortlisted and subsequently selected. Applications can also be rejected according to the defined workflow. When a student is selected, the student's placement information is updated. The seventh module is **Reporting and Statistics**. The system generates information about placed and unplaced students, placement count by branch, average package by branch, company-wise selections, top package students, and the overall placement percentage.

Non-Functional Requirements

The system is designed with several non-functional requirements to improve its reliability, maintainability, usability, and correctness.

Usability is an important requirement because users should be able to operate the system through a clear console menu. The application provides numbered options for common operations such as adding students, adding companies, creating placement drives, applying to drives, checking eligibility, processing applications, and viewing reports.

Reliability is achieved by validating important operations and preventing invalid application states. For example, a student cannot apply to the same placement drive more than once, an application cannot be shortlisted unless it is eligible, and a student cannot apply to a closed drive. These business rules reduce inconsistent system states.

Maintainability is supported by separating the application into logical sections such as models, repositories, services, and the main application interface. The model classes represent data entities, repositories manage stored objects, and service classes contain business logic.

Error handling is another requirement of the system. Invalid input, missing records, duplicate applications, closed placement drives, and invalid application status transitions are handled through validation and exceptions. The main program also catches exceptions and displays appropriate error messages to the user.

Performance is supported by using Java collection classes such as HashMap, LinkedHashMap, HashSet, ArrayList, and stream operations for searching, filtering, sorting, grouping, and calculating statistics.

System Architecture

The Student Placement Management System follows a logical layered architecture consisting of the presentation layer, service layer, repository layer, and model layer. Although the current implementation is contained in a single Java source file for simplicity, the code is logically organized into separate responsibilities.

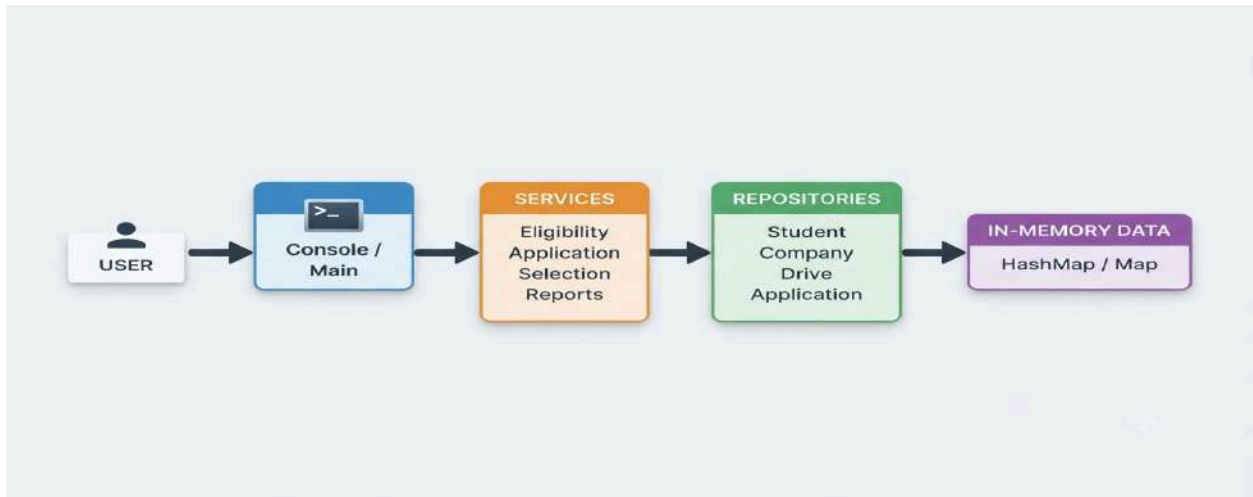
The presentation layer is represented by the main class and console menu. It accepts user input using the Scanner class and invokes the appropriate operations based on the selected menu option.

The model layer contains the core entities of the system, including Student, Company, PlacementDrive, and Application. These classes store the information associated with each real-world entity and provide methods for accessing and modifying their state.

The repository layer acts as an in-memory data storage mechanism. StudentRepository, CompanyRepository, DriveRepository, and ApplicationRepository use Java Map collections to store and retrieve objects. These repositories provide operations such as save, findById, findAll, and findByStudentId.

The service layer contains the main business logic. EligibilityService evaluates whether students satisfy placement criteria, ApplicationService manages application submission and validation, SelectionService manages shortlisting, selection, and rejection, and ReportService generates placement statistics.

The overall data flow is therefore: user input is received by the console interface, the appropriate service performs the required business operation, repositories provide access to stored data, and the resulting information is displayed back to the user.



DESIGN DECISIONS AND RATIONALE

The project was designed using object-oriented programming principles because the placement process naturally consists of real-world entities such as students, companies, placement drives, and applications. Creating separate classes for these entities makes the system easier to understand and maintain.

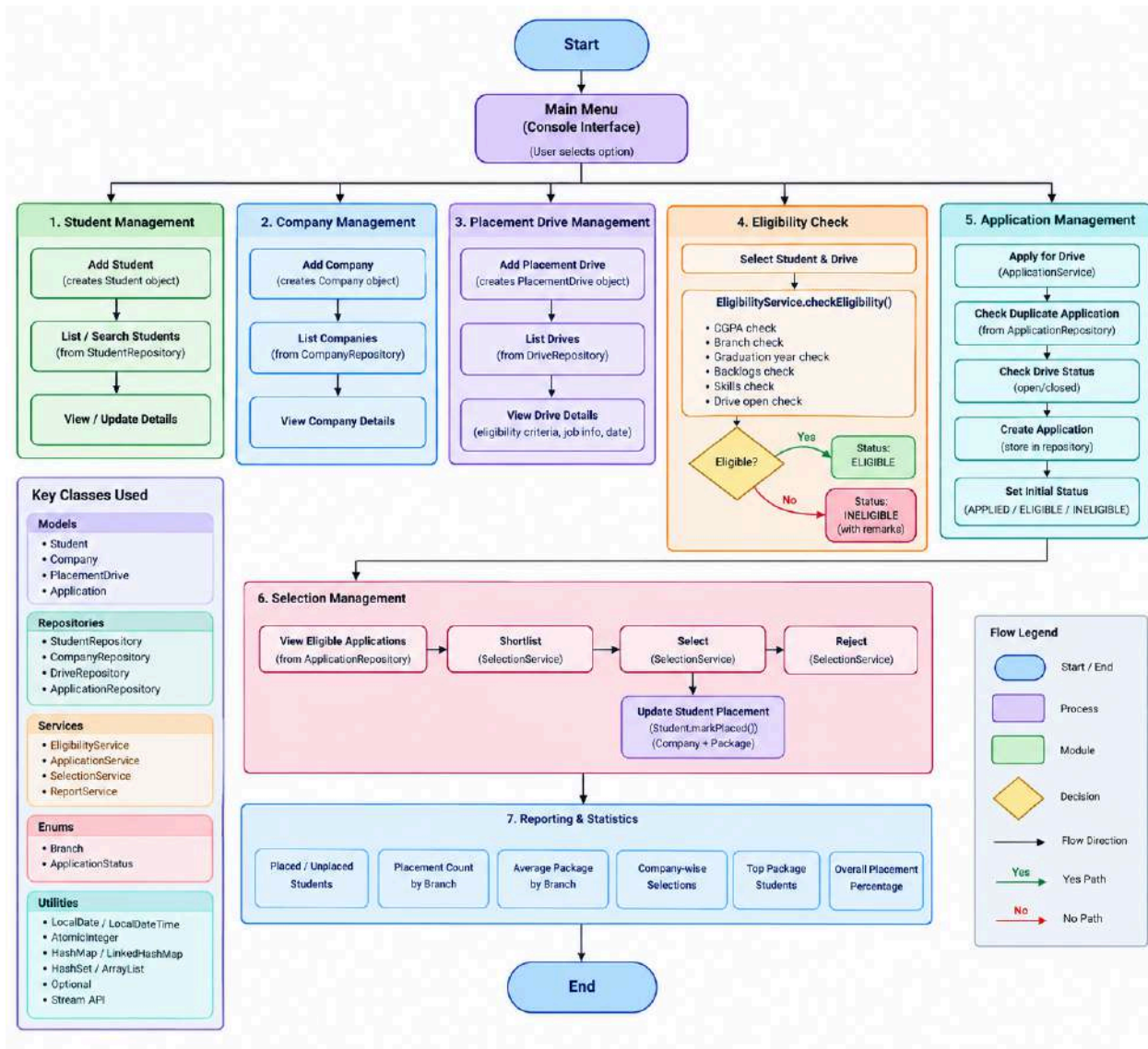
Encapsulation was used by keeping important attributes private inside the model classes and providing controlled methods for accessing or modifying them. For example, student placement information is updated through methods such as `markPlaced` rather than allowing unrestricted modification of the object's internal state.

Enumerations were used for values that have a fixed set of possible states. The `Branch` enumeration defines supported academic branches, while `ApplicationStatus` represents the different stages of an application such as **APPLIED, ELIGIBLE, INELIGIBLE, SHORTLISTED, SELECTED, and REJECTED**.

The repository pattern was selected to separate data storage operations from business logic. The service classes therefore do not directly manipulate the underlying storage structure. This separation makes the application easier to modify if the in-memory storage is later replaced with a relational database using JDBC or JPA.

The service-based design was selected to keep business rules centralized. Eligibility rules are handled by `EligibilityService`, application rules by `ApplicationService`, selection transitions by `SelectionService`, and reporting calculations by `ReportService`. This reduces duplication and improves maintainability. Java collection classes were selected because the current application operates on in-memory data. Maps provide efficient lookup using unique identifiers, sets are useful for skills and eligibility criteria, and lists are useful for returning collections of students and applications. Java streams are used for filtering, grouping, sorting, and calculating statistics.

Design Diagrams



IMPLEMENTATION DETAILS

The Student Placement Management System is implemented in Java as a console-based application. The program uses the Scanner class to receive user input and a menu-driven interface to provide access to the system's major operations.

The model layer contains Student, Company, PlacementDrive, and Application classes. Student stores academic and skill information, Company stores recruiter information, PlacementDrive stores job and eligibility information, and Application represents a student's application to a particular placement drive. The system also uses Branch and ApplicationStatus enumerations for controlled values.

The repository layer consists of StudentRepository, CompanyRepository, DriveRepository, and ApplicationRepository. These classes use Map-based in-memory storage to maintain objects and provide operations for saving, retrieving, and listing records. ApplicationRepository additionally supports searching applications belonging to a particular student.

The EligibilityService implements the eligibility algorithm. It checks minimum CGPA, allowed branch, graduation year, maximum backlogs, required skills, and whether the drive is closed. If any criterion is not satisfied, the service records the reason and returns an unsuccessful eligibility result.

The ApplicationService manages application submission. It prevents duplicate applications, prevents applications to closed drives, and applies a business rule for students who are already placed. It then creates an Application object and assigns either ELIGIBLE or INELIGIBLE status based on the eligibility result.

The SelectionService manages the application lifecycle. Eligible applications can be shortlisted, shortlisted applications can be selected, and eligible or shortlisted applications can be rejected. When a student is selected, the student's placement status, selected company, and package are updated.

The ReportService performs data analysis using Java Stream API operations. It generates lists of placed and unplaced students, placement counts by branch, average package by branch, company-wise selection counts, top package students, and the overall placement percentage.

The application also uses Java date and time classes such as LocalDate and LocalDateTime for placement drive dates and application timestamps.

AtomicInteger is used by the ID generator to create sequential identifiers for students, companies, drives, and applications.

WORKING SCREENSHOTS :

```
==== LIBRARY MANAGEMENT SYSTEM ====
1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit
Enter your choice: 2
ID    Title           Author      Available/Total
1     Pride and Prejudice  Jnae Austin  20/20
2     Wuthering Heights   Emily Bronte  10/10

==== LIBRARY MANAGEMENT SYSTEM ====
1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit
Enter your choice: 3
Enter title or part of title to search: Pride
1     Pride and Prejudice  Jnae Austin  20/20

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit
Enter your choice: 4
Enter member name: Ram
Enter 10 digit phone number: 985562741
Invalid input. Enter exactly 10 digits, try again.
Enter 10 digit phone number: 9855627411
Member added successfully with ID: 1

===== LIBRARY MANAGEMENT SYSTEM =====
1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit
Enter your choice: 5
ID      Name      Phone
1       Ram      9855627411
```

```
===== LIBRARY MANAGEMENT SYSTEM =====
```

1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit

Enter your choice: 6

Enter book ID to issue: 2

Enter member ID: 1

Book "Wuthering Heights" issued to Ram

Allowed period is 14 days. Fine after that is Rs 5/day.

```
===== LIBRARY MANAGEMENT SYSTEM =====
```

1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit

Enter your choice: 8

Book ID	Member ID	Status	Fine
2	1	Returned	0

```
===== LIBRARY MANAGEMENT SYSTEM =====
```

1. Add Book
2. View All Books
3. Search Book by Title
4. Add Member
5. View All Members
6. Issue Book
7. Return Book
8. View Issued Books
9. Exit

Enter your choice: 9

Thank you. Closing library system.

Choose an option: 7

Student[STU-1] Ellur Gowrav | AIML | GradYear:2029 | CGPA:8.92 | Backlogs:0 | Skills:[JAVA, CSS, HTML, PYTHON] | Placed:NO

Enter Student ID (or press Enter to cancel): STU-1

Drive[DRV-1] AI engineer @ MICROSOFT | Package:56.00 LPA | Date:2026-09-16 | MinCGPA:9.10 | MaxBacklogs:0 | Branches:[MECH, CSE, AIML] | GradYears:[2028, 2029] | Skills:[JAVA, PYTHON] | OPEN

Enter Drive ID (or press Enter to cancel): DRV-1

Application result: Application[APP-1] Ellur Gowrav -> AI engineer (MICROSOFT) | Status:INELIGIBLE | Remarks:[CGPA 8.92 is below required minimum 9.10]

TESTING APPROACH

Testing was performed by executing the major functional operations of the Student Placement Management System and verifying whether the system produced the expected output. The testing approach focused on functional validation, input validation, business-rule validation, and application-state validation.

Student management was tested by adding students with valid information and displaying the stored student records. Company management was tested by adding companies and verifying that company information could be retrieved correctly. Placement drive creation was tested using different eligibility requirements such as CGPA, branch, graduation year, backlog limit, and required skills.

The eligibility module was tested using students who both satisfy and fail different eligibility conditions. The expected result was that eligible students receive an ELIGIBLE result, while students who fail one or more conditions receive an INELIGIBLE result with appropriate reasons.

Application management was tested for successful applications, duplicate applications, applications to closed drives, and applications by already placed students. Selection management was tested by verifying that only applications in valid states could be shortlisted or selected.

The reporting module was tested by comparing the generated statistics against the stored student and application information. This included placement percentage, placement count by branch, average package, company-wise selections, and top package students.

Exception and validation testing was also performed on invalid menu choices, invalid identifiers, invalid CGPA values, invalid branch values, and invalid application state transitions.

Challenges Faced

One of the main challenges during development was designing the placement workflow so that applications could move through valid states without creating inconsistent data. The system therefore uses `ApplicationStatus` values and validation methods to control transitions between application states.

Another challenge was implementing eligibility checking with multiple conditions. A placement drive may specify requirements for CGPA, branch, graduation year, backlogs, and skills simultaneously. The `EligibilityService` was designed to evaluate each condition independently and record the reasons when a student does not satisfy the requirements.

Managing relationships between objects was another challenge. An `Application` must maintain references to both a `Student` and a `PlacementDrive`, while the `PlacementDrive` must also be associated with a `Company`. Maintaining these relationships correctly was important for application processing and report generation.

Generating useful placement statistics also required filtering, grouping, sorting, and aggregation of collection data. Java Stream API operations were used to simplify these calculations.

A further challenge was maintaining a clear separation between data storage and business logic. The repository classes were introduced to isolate storage operations, while service classes handle business rules. This makes the current in-memory implementation easier to extend with persistent database storage in the future

Learnings & Key Takeaways

The project provided practical experience in applying Java object-oriented programming concepts to a real-world problem. The development process demonstrated how real-world entities can be represented using classes and how their relationships can be modeled using object references.

A major learning outcome was understanding encapsulation and the importance of protecting object data through private fields and controlled methods. The project also strengthened understanding of constructors, enumerations, collections, method overriding, and static nested classes.

The project provided practical experience with Java Collections Framework classes such as Map, List, ArrayList, HashMap, LinkedHashMap, HashSet, and Set. The Java Stream API was also used to perform filtering, sorting, grouping, counting, and averaging operations.

Exception handling and validation were important learning areas because the application needs to handle invalid input and invalid business operations. The project demonstrated how exceptions can be used to prevent incorrect application states.

The project also improved understanding of software design principles. Separating models, repositories, and services showed how different responsibilities can be organized within a larger application. Overall, the project helped bridge the gap between learning individual Java concepts and combining them to create a functional application.

Future Enhancements

The current system can be extended in several ways to make it more suitable for real-world deployment. The first major enhancement would be replacing the current in-memory repositories with a relational database such as MySQL or PostgreSQL. JDBC could be used for direct database connectivity, while JPA could be introduced to provide object-relational mapping and simplify database CRUD operations.

A graphical user interface or web-based interface could also be added to replace the current console-based interface. This would make the system easier for students, placement coordinators, and recruiters to use.

Authentication and role-based access control could be implemented so that students, placement officers, and company representatives have different permissions. Students could manage their profiles and applications, while placement officers could manage drives and selections.

The system could also include automated notifications for important events such as new placement drives, eligibility results, interview schedules, and selection results. Email or notification services could be integrated for this purpose.

Additional reporting and analytics could be introduced, including placement trends by academic year, company-wise hiring trends, skill demand analysis, package distributions, and department-wise placement performance.

Finally, the system could be deployed as a web application with a REST API and a frontend interface. This would allow students and placement officers to access the system remotely and would make the project more scalable.

REFERENCES AND RESEARCH

The development of the Placement System is based primarily on the Java programming concepts covered in the course and the implementation contained in the project source code. The project applies concepts related to Java classes and objects, constructors, encapsulation, collections, exception handling, strings, control flow, input/output, static members, and constants.

Java documentation and course learning material can also be referred to for understanding the syntax and concepts used in the implementation, including Java classes and objects, constructors, access control, collections such as ArrayList, exception handling, the Scanner class, strings, loops, conditional statements, and switch-case statements.

The project is currently a console-based, in-memory Java application. Database technologies such as JDBC and JPA, graphical user interfaces, and persistent storage are considered future enhancements rather than features of the current implementation.